

Symmetry Reduction in the Detailed Balance Checker: Feasibility, Expected Speedup, and Correctness Conditions

1 What the method proposes

The detailed balance (DB) checker must verify $T(s_i \rightarrow s_j) \pi(s_i) = T(s_j \rightarrow s_i) \pi(s_j)$ for every pair of states (s_i, s_j) found by BFS. For a component of N states this is $N(N-1)/2$ pairs. At `NGrid=3` ($N = 504$) that is 126,756 pairs; at `NGrid=6` ($N \approx 100,000$) it is $\sim 5 \times 10^9$, which is infeasible.

The proposed speedup exploits the fact that for a translation-invariant algorithm on a periodic lattice, all nine translated copies of a configuration pair produce *identical physics*. If the pair (s, s') satisfies DB, then every translated copy $(g \cdot s, g \cdot s')$ automatically satisfies DB too, by a three-line argument:

$$T(g \cdot s \rightarrow g \cdot s') \pi(g \cdot s) = T(s \rightarrow s') \pi(s) = T(s' \rightarrow s) \pi(s') = T(g \cdot s' \rightarrow g \cdot s) \pi(g \cdot s').$$

The outer equalities use the symmetry of T and π ; the middle uses DB for the original pair. The reverse direction holds identically: a violation for (s, s') implies a violation for every translated copy. Therefore it suffices to check *one pair per family of equivalent pairs*.

For a 3×3 lattice with 9 translations ($|G| = 9$), the 126,756 pairs reduce to $126,756/9 = 14,084$. Adding the 8 rotations and reflections of the square (D4, giving $|G| = 72$) reduces this further to $\approx 1,760$ pairs.

2 Where the runtime actually comes from

Understanding whether the speedup is real requires understanding the current bottleneck precisely.

The checker pipeline is: (1) BFS to find all N states; (2) for each pair with non-zero T , build the symbolic DB expression; (3) deduplicate expressions by hash and run **FastChecker** on each unique one; (4) run **FullSimplify** on expressions **FastChecker** could not resolve; (5) scan for violations.

Step (3) uses Mathematica's `Hash[]` on the symbolic expression object. This matches *syntactically identical* expressions — same atoms in the same positions — not merely mathematically equivalent ones. G-equivalent pairs (e.g. the same cluster geometry translated to a different lattice site) produce expressions that reference different site indices, different neighbor lists, and different random-variable assignments; they come out in different symbolic forms even though their mathematical content is the same. They therefore hash differently, and the deduplication step does *not* group them.

Consequence: the $\sim 3,000$ unique expressions reported for `NGrid=3` are genuinely distinct after all caching. The orbit reduction would reduce this count by the full factor of $|G|$, not a fraction of it.

Step (4), the `FullSimplify` calls, is the runtime bottleneck. For `vmc.lattice.w1` (polynomial conditions, no `Erfc`), **FastChecker** resolves most expressions quickly, but a fraction

— roughly 10 % of the unique expressions, or ~ 300 at `NGrid=3` — require `FullSimplify`, each taking ~ 0.5 s. At 300 calls $\times$ 0.5 s, spread across 4 kernels, total time is ≈ 3 minutes, consistent with observed runtimes.

3 Quantitative impact

All rows below refer to the 504-state connected component found at `NGrid=3`. The `FullSimplify` estimate assumes $\sim 10\%$ of unique expressions require it and ~ 0.5 s each.

	No symmetry	Translations ($ G =9$)	D4+trans. ($ G =72$)
Pairs to check	126,756	14,084	$\approx 1,760$
Unique sym. exprs	$\sim 3,000$	~ 333	~ 42
<code>FullSimplify</code> calls	~ 300	~ 33	~ 4
Estimated runtime	3 min	~ 20 s	~ 2 s

These estimates are approximate. The `FullSimplify` fraction depends on the specific expressions generated, which varies with `NGrid` and the seed configuration. The table should be read as order-of-magnitude guidance, not a performance guarantee.

For `NGrid=6` the reduction from 5×10^9 to $\approx 1.7 \times 10^7$ pairs with $|G|=288$ is the difference between infeasible and potentially tractable. The absolute runtime at that scale is harder to predict without profiling, since it depends on the fraction of expressions needing `FullSimplify`, which may differ from the `NGrid=3` case.

4 Conditions for correctness, and their risks

The reduction is logically valid but depends on conditions that must be verified, not assumed. A failure in any one of them can cause the checker to report PASS on a broken algorithm.

1. The algorithm must actually be symmetric (critical). The entire reduction assumes $T(g \cdot s \rightarrow g \cdot s') = T(s \rightarrow s')$. If this fails, families will contain pairs with different DB status, and the checker will certify a violating pair based on a non-violating representative. `vmmc.biased.wl` demonstrates this exactly: its rightward bias means a pair involving a rightward cluster move violates DB, but the 90° -rotated copy (an upward move, which is unbiased) satisfies DB. With D4 symmetry assumed, the upward pair becomes the representative, and the checker incorrectly returns PASS. Section 4 below describes how to verify symmetry from the source code.

2. The connected component must be closed under the symmetry. Every symmetry image $g \cdot s$ of every state s in the component must also be in the same component. If not, the effective symmetry group for that component is smaller than G , and the claimed speedup factor is wrong. For `NGrid=3` with $|G|=9$, the component size $504 = 56 \times 9$ is consistent with this, but it should be confirmed programmatically, not inferred from arithmetic.

3. BFS must not be short-circuited using canonical states. A tempting shortcut is to run BFS only from one representative state per symmetry family and check only pairs of representatives. This is wrong. The canonical representative of s is $g_0 \cdot s$ and the canonical representative of s' is $g_1 \cdot s'$ for (generically) different $g_0 \neq g_1$. The transition $T(g_0 \cdot s \rightarrow g_1 \cdot s')$ is not equal to $T(s \rightarrow s')$ in general. BFS must run on all N states, and orbit reduction only controls which pairs are sent to the DB checker.

4. Canonical representatives must be computed per pair. Some configurations are self-symmetric (a non-trivial g fixes the state $g \cdot s = s$), making their family smaller than $|G|$. The implementation must compute the canonical representative explicitly for every pair, not

assume all families have size $|G|$ and skip by that factor.

5. Rotation formulas must be correct for the lattice size. The 90° rotation $(r, c) \mapsto (c, L-1-r)$ maps lattice sites to lattice sites for any L , but an off-by-one error produces a mapping that is not a true bijection. This bug is silent: the code runs without error but computes wrong orbit representatives, potentially skipping pairs that were never actually certified.

5 Verifying position-independence from source code

The symmetry reduction can only be trusted if the algorithm file is genuinely position-independent. For `vmmc_lattice.wl`-style algorithms, this can be checked by reading three specific parts of the source code.

Seed selection. Look for the line that chooses the seed particle. It should be `RandomChoice[occupied]` where `occupied` is the list of all non-empty sites, with no weighting by position. Any position-dependent weighting (e.g. preferring sites near the lattice centre) would break translation invariance.

Displacement selection. Look for the line that chooses the proposed move direction. For translation invariance it must be `RandomChoice[$displacements]` where `$displacements` is a fixed list that does not depend on the current state or on which site was seeded. For D4 invariance additionally, the list must be closed under rotation and reflection: the uniform box $\{(\pm 1, 0), (0, \pm 1), (\pm 1, \pm 1)\}$ satisfies this; any list that over-represents one direction (as in `vmmc.biased.wl`) does not.

Energy computation. Find the function that computes pair interaction energy. It should call `couplingJ[a, b, d2]` where `d2` is the squared *periodic* distance between the two sites:

$$d^2 = \min(\Delta r, L - \Delta r)^2 + \min(\Delta c, L - \Delta c)^2,$$

with $\Delta r = |r_1 - r_2|$ and $\Delta c = |c_1 - c_2|$. The distance formula must use the minimum-image convention (the `min` terms) to be periodic-boundary-correct. If the energy instead uses raw coordinate differences $\Delta r = r_1 - r_2$ (not wrapped), it will not be translation-invariant at the lattice boundary. For D4 invariance, note that 90° rotation swaps Δr and Δc and negates one; since both appear squared and summed, d^2 is unchanged.

These three checks can be performed by reading fewer than 20 lines of any algorithm file. They are necessary but not sufficient: they confirm that the *stated logic* is symmetric, but not that the supporting library functions (`$neighborsD2`, `$applyDir`, etc.) implement that logic correctly. Those library functions should be independently verified once and their symmetry properties documented.

6 Is it worth implementing?

At NGrid=3 (current scale): Yes, the speedup is real. The hash-based deduplication does not capture G-equivalence, so the orbit reduction genuinely reduces `FullSimplify` calls from ~ 300 to ~ 4 with D4+translations, cutting 3 minutes to ~ 2 seconds. However, 3 minutes is already tolerable for a correctness check. The implementation requires satisfying five non-trivial correctness conditions, and a bug in any one of them introduces silent false negatives. The benefit at this scale does not clearly outweigh the implementation risk.

At NGrid=6 and beyond: The checker is entirely infeasible without orbit reduction. 5×10^9 pairs cannot be constructed, let alone evaluated, in any reasonable time. The orbit reduction to $\sim 1.7 \times 10^7$ pairs is not an optimisation; it is a prerequisite for the checker to run at all. At this scale the implementation is worth doing despite the complexity.

Recommendation: Do not implement for `NGrid=3`. Implement — with all five correctness conditions satisfied explicitly — when `NGrid=5` or higher is a target.